

School Enrollment Management System

A Console-Based C++ Application with SQLite3 Database Integration

Academic Paper · School Management System Project · June 2026

ABSTRACT

This paper presents the design, implementation, and evaluation of a School Enrollment Management System developed using C++ with object-oriented programming principles and an SQLite3 relational database backend. The system addresses the administrative challenges of managing student enrollments, academic sessions, and user authentication in an educational environment. It provides two distinct access roles — USER and ADMIN — each with specific capabilities aligned to their responsibilities. The USER role enables students or representatives to enroll students into available sessions by selecting from displayed sections and modalities. The ADMIN role allows administrators to manage sessions, monitor enrollment data, and maintain system records. Key features include input validation, duplicate prevention, role-based access control, and a session-slot management mechanism that automatically adjusts available slots upon enrollment or removal. The system is implemented as a cross-platform console application compiled via the MSYS2 UCRT64 toolchain on Windows, with data persistently stored in a local SQLite3 database file.

Keywords:

C++, Object-Oriented Programming, SQLite3, Enrollment System, Role-Based Access Control, Console Application, Student Information System

1. INTRODUCTION

The management of student enrollment records is a critical function in educational institutions. Traditional paper-based or spreadsheet approaches are prone to errors, lack real-time validation, and make it difficult to enforce business rules such as slot capacity limits or duplicate prevention. The School Enrollment Management System (SEMS) was developed to address these limitations through a structured, database-driven application.

Built entirely in C++ using object-oriented design patterns, the system leverages SQLite3 as a lightweight, serverless relational database engine, making it deployable without additional infrastructure. The application enforces a clear separation of concerns: the DatabaseConnection class manages all database interactions, while domain classes (User, Session, Student) encapsulate their respective business logic. The SchoolSystem class serves as the controller, coordinating user interaction, authentication, and menu-driven navigation.

This paper documents the system's architecture, features, data model, validation rules, and overall flow, providing a comprehensive reference for understanding its design decisions and implementation strategies.

2. OBJECTIVES

The primary objectives of the School Enrollment Management System are:

- To provide a secure, role-based authentication mechanism distinguishing USER and ADMIN access levels.
- To enable students or authorized users to enroll in available academic sessions with real-time slot availability checking.
- To allow administrators to create, update, and delete sessions, with automatic enforcement of enrollment capacity constraints.
- To maintain data integrity through input validation, uniqueness constraints, and foreign key enforcement in the database layer.
- To ensure a consistent and recoverable user experience through a retry mechanism on login failures and graceful handling of invalid inputs.
- To provide a persistent, file-based data store using SQLite3 that survives application restarts.

3. SYSTEM ARCHITECTURE & DESIGN

The system follows a layered architecture consisting of a Presentation Layer (console I/O via SchoolSystem), a Business Logic Layer (domain classes: User, Session, Student), and a Data Access Layer (DatabaseConnection singleton and raw SQLite3 API calls). This separation ensures maintainability and clear responsibility boundaries.

Core Classes:

- DatabaseConnection — Singleton. Manages SQLite3 connection lifecycle, executes SQL statements, and provides prepared statement support. Includes executeSilent() for suppressing expected migration errors.
- User — Manages user accounts, authentication (plain-text password comparison), role assignment (USER/ADMIN), and registration validation.
- Session — Manages academic session records including name, section, total/available slots, and learning modality. Provides static helpers: getAll(), findById(), getAvailableSections(), sessionNameExists().
- Student — Manages student enrollment records with fields: student_number (unique), student_name, gender, age, section, session_id (FK), learning_modality. Handles enroll() which decrements session slots transactionally, and remove() which restores them.
- SchoolSystem — Central controller. Initializes the database, drives the main loop, handles login/register, and routes to role-specific menus.

The system initializes via CREATE TABLE IF NOT EXISTS statements on every startup, ensuring idempotent schema creation. ALTER TABLE migration statements for gender and age columns are executed silently to support existing databases.

4. KEY FEATURES

4.1 Authentication & Access Control

Login enforces a maximum of 3 attempts, providing specific error messages for empty inputs, unknown usernames, and incorrect passwords. After 3 failures, the system returns to the main menu. Role detection after successful login routes the user to the appropriate menu (USER or ADMIN).

4.2 User Registration

New users may register with a username, password (minimum 6 characters), and email address. Uniqueness is validated against existing records for both username and email before insertion.

4.3 Session Management (ADMIN)

Administrators can add new sessions specifying name, section, total slots, and learning modality. Updates allow partial modification (blank input retains current value) with a safeguard preventing total slots from being set below the current enrolled count. Deletion triggers a cascade warning and requires confirmation, after which all enrolled students in the session are also removed via SQLite3's ON DELETE CASCADE foreign key constraint.

4.4 Student Enrollment (USER)

The enrollment flow collects a unique student number (with duplicate check), full name, gender, and age (validated 1–120). The user then selects from a dynamically generated list of available sessions showing section, modality, available slots, and session name. Upon selection, section and modality are automatically assigned from the chosen session, preventing mismatches. Enrollment is performed within a database transaction that inserts the student record and decrements the session's available_slots atomically.

4.5 Student Removal (ADMIN)

Administrators can remove any enrolled student after viewing the full roster. Removal requires explicit confirmation and restores the corresponding session's available_slot count by 1.

5. DATABASE SCHEMA

Table: users

- id INTEGER PRIMARY KEY AUTOINCREMENT
- username TEXT UNIQUE NOT NULL
- password TEXT NOT NULL
- email TEXT UNIQUE NOT NULL
- role TEXT DEFAULT 'USER' CHECK(role IN ('USER', 'ADMIN'))
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

Table: sessions

- id INTEGER PRIMARY KEY AUTOINCREMENT
- session_name TEXT UNIQUE NOT NULL
- section TEXT NOT NULL
- total_slots INTEGER NOT NULL
- available_slots INTEGER NOT NULL
- learning_modality TEXT NOT NULL (Face-to-Face / Online / Hybrid)
- created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

Table: students

- id INTEGER PRIMARY KEY AUTOINCREMENT
- student_number TEXT UNIQUE NOT NULL
- student_name TEXT NOT NULL
- gender TEXT NOT NULL DEFAULT "
- age INTEGER NOT NULL DEFAULT 0
- section TEXT NOT NULL
- session_id INTEGER NOT NULL REFERENCES sessions(id) ON DELETE CASCADE
- learning_modality TEXT NOT NULL
- enrolled_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP

6. INPUT VALIDATION SUMMARY

- All text fields: checked for empty or whitespace-only strings before processing.
- Student Number: checked for uniqueness against the students table before insertion.
- Age: must be a numeric value in the range 1–120.
- Password: minimum length of 6 characters enforced at registration.
- Session slots: total slots must be > 0 on creation; update cannot reduce below current enrollment.
- Session ID selection: validated against the returned list index; out-of-range selections are rejected.
- Login: maximum 3 consecutive attempts with per-error messages (empty, not found, wrong password).

7. IMPLEMENTATION ENVIRONMENT

- Language: C++11 standard
 - Compiler: GCC 16.1.0 via MSYS2 UCRT64 toolchain (g++.exe)
 - Database Engine: SQLite3 (bundled with MSYS2 ucrt64 package)
 - Build Command: g++ -std=c++11 -linclude src/*.cpp -lsqlite3 -o build/school_management.exe
 - Platform: Windows 10/11 (console application)
 - DB File: school_management.db (created in the same directory as the executable)
-

8. CONCLUSION

The School Enrollment Management System demonstrates how fundamental object-oriented programming principles — encapsulation, abstraction, and separation of concerns — can be applied to build a functional and maintainable administrative application. The integration of SQLite3 provides persistent, transactional data storage without the complexity of a full database server, making it suitable for small-to-medium institutional deployments.

The system successfully addresses its design objectives: secure role-based access, enforced data integrity, real-time slot management, and user-friendly input validation. Future enhancements could include password hashing for improved security, a graphical user interface, network-based multi-user support, and report generation capabilities such as enrollment summaries and session occupancy reports.

REFERENCES

-
- [1] SQLite Consortium. (2024). SQLite Documentation. <https://www.sqlite.org/docs.html>
 - [2] Stroustrup, B. (2013). The C++ Programming Language (4th ed.). Addison-Wesley.
 - [3] Deitel, P., & Deitel, H. (2017). C++ How to Program (10th ed.). Pearson Education.
 - [4] MSYS2 Project. (2024). MSYS2 Documentation. <https://www.msys2.org/docs/>
 - [5] ISO/IEC 14882:2011. (2011). Programming Language C++ (C++11 Standard). ISO.

